Unit 3：Portfolio Optimization

3.1：Markowitz 均值-方差模型

给定 n 支股票，在预算 B 内决定每支股票的投资金额 x_i，使投资组合方差最小，同时期望收益不低于目标 \beta，且不允许卖空。

参数：

i：股票，$i = 1,2,\ldots,n$　　μ_i：股票 i 的期望收益

σ_{ij}：股票 i 与 j 的协方差（Σ 为协方差矩阵）

B：投资预算　　β：目标期望收益

目标函数： $\min \quad \sum_{i=1}^{n}\sum_{j=1}^{n}\sigma_{ij}\,x_i x_j \;=\; x^{\top}\Sigma x$

$$\sum_{i=1}^{n} x_i \le B \qquad \text{（预算约束）}$$

约束条件：

$$\sum_{i=1}^{n}\mu_i\,x_i \ge \beta \qquad \text{（收益目标）}$$

$$x_i \ge 0, \quad \forall i \qquad \text{（无卖空）}$$

```
import pandas as pd
import gurobipy as gp

# ===== 数据读取=====
data = pd.read_csv('data_portfolio.csv')


# 读取CSV文件，数据行为各期收益，列为各股票
stocks = data.columns.values # 提取列名（股票代码列表）

ret_mean = data.mean()# 对每列求均值 → 各股票的期望收益率
mu = dict(ret_mean) # 转成字典，键=股票名，值=期望收益
Sigma = data.cov().values # 计算各列的协方差矩阵（即各股票收益的协方差）

budget = 1000
beta = 5
# ===== 模型建立 =====
m = gp.Model('markowitz')

x = m.addVars(stocks, lb=0, name='x')

# 目标：min x^T Sigma x
m.setObjective(
    gp.quicksum(Sigma[i, j] * x[stocks[i]] * x[stocks[j]]
                for i in range(len(stocks)) for j in range(le
n(stocks))),
    gp.GRB.MINIMIZE
)

# 约束1：预算
m.addConstr(gp.quicksum(x[s] for s in stocks) <= budget, name
='budget')
# 约束2：收益目标
m.addConstr(gp.quicksum(mu[s] * x[s] for s in stocks) >= bet
a, name='return')
# 求解
m.optimize()
# 输出结果
if m.status == gp.GRB.OPTIMAL:
    print(f"最优目标值（最小方差）= {m.ObjVal:.6f}")
    print("\n各股票投资金额：")
    for s in stocks:
        if x[s].X > 1e-3:
            print(f"  {s}: {x[s].X:.4f}")
    print(f"\n总投资：{sum(x[s].X for s in stocks):.4f}")
    print(f"总期望收益：{sum(mu[s] * x[s].X for s in stocks):.
4f}")
```

4.3：Fixed-Charge Facility Location（反面）

每个候选设施点 j 开设需要支付固定成本 f_j，且有容量限制 v_j。客户 i 的需求为 d_i，由设施 j 服务每单位需求的可变成本为 c_{ij}。决定开设哪些设施以及如何分配客户需求，使总成本（固定成本 + 可变成本）最小。

数学建模

参数：

d_i：客户 i 的需求

f_j：开设设施 j 的固定成本

v_j：设施 j 的容量

c_{ij}：单位服务成本

决策变量：

$x_j \in \{0,1\}$：设施 j 是否开设

$y_{ij} \in [0,1]$：客户 i 由设施 j 服务的**需求比例**

目标函数：

$$\min \quad \sum_{j\in J} f_j\,x_j + \sum_{i\in I}\sum_{j\in J} c_{ij}\,d_i\,y_{ij}$$

约束条件：

$$\sum_{j\in J} y_{ij} = 1, \quad \forall i \in \mathcal{I} \qquad \text{（需求完全满足）}$$

$$\sum_{i\in I} d_i\,y_{ij} \le v_j x_j, \quad \forall j \in \mathcal{J} \qquad \text{（容量约束）}$$

$$x_j \in \{0,1\}, \quad y_{ij} \in [0,1]$$

3.2：Portfolio Optimization II — 最大化达到目标收益的概率

在同样的投资决策框架下，不再最小化方差，而是最大化投资组合收益不低于目标 β 的概率。在正态分布假设下，可转化为一个可求解的优化问题。

假设股票收益 $\vec{r}$ 服从多元正态分布，则：

$$\mathbb{P}[\vec{r}^{\top}x \ge \beta] = \Phi\left(\frac{\boldsymbol{\mu}^{\top}\boldsymbol{x} - \beta}{\sqrt{\boldsymbol{x}^{\top}\Sigma\boldsymbol{x}}}\right)$$

最大化该概率等价于最大化括号内的值（Sharpe ratio 形式）。进一步变量替换：令 $y = \boldsymbol{\mu}^{\top}\boldsymbol{x} - \beta$，$\bar{\boldsymbol{x}} = \boldsymbol{x}/y$，$z = 1/y$，则得：

数学建模（转化后的二次优化问题）

决策变量： $\bar{x}_i$：辅助变量（与 x_i 成正比）；$z \ge 0$：辅助变量

目标函数： $\min \quad \bar{x}^{\top}\Sigma\bar{x}$

$$\sum_{i=1}^{n}\bar{x}_i \le B \cdot z \qquad \text{（预算约束）}$$

约束条件：

$$\sum_{i=1}^{n}\mu_i\,\bar{x}_i - \beta \cdot z = 1 \qquad \text{（归一化约束）}$$

$$\bar{x}_i \ge 0, \quad z \ge 0$$

还原实际投资金额： $x_i = \frac{\bar{x}_i}{z}$

```
import pandas as pd
from gurobipy import *

# ===== 数据读取 =====
data1 = pd.read_csv('data_portfolio.csv')
stocks = data1.columns.values

ret_mean = data1.mean()
ret_mean = dict(ret_mean)
ret_cov = data1.cov()
budget = 1000
exp_ret = 5
# ===== 模型建立 =====
m = Model('portfolio_max_prob')

x = m.addVars(stocks, name='bar')
z = m.addVar(name='z', lb=0)
# 目标函数：min x^T Σ x
m.setObjective(
    quicksum(ret_cov.loc[i, j] * x[i] * x[j]
             for i in stocks for j in stocks),
    sense=GRB.MINIMIZE
)
# 约束1：预算约束
m.addConstr(
    quicksum(x[i] for i in stocks) <= budget * z,
    name='budget_con'
)
# 约束2：归一化约束
m.addConstr(
    quicksum(ret_mean[i] * x[i] for i in stocks) - exp_ret * z == 1,
    name='norm_con'
)
m.write('model.lp')
m.optimize()
# ===== 结果输出 =====
if m.status == GRB.OPTIMAL:
    z_val = z.X
    if abs(z_val) < 1e-6:
        print("Warning: z 接近 0，模型可能无意义或 β 设置过高")
    else:
        print(f"\n最优目标值 (x^T Σ x) = {m.ObjVal:.6f}")
        print(f"辅助变量 z = {z_val:.6f}")
        print("\n--- 实际投资金额 x = x̄/ z ---")

        total_invested = 0
        actual_x = {}
        for s in stocks:
            val = x[s].X / z_val
            actual_x[s] = val
            total_invested += val
            if abs(val) > 1e-3:
                print(f"{s}: x̄= {x[s].X:.4f},  x = {val:.4f}")
```

4.5：Exercise — p-Median 中国城市设施选址（反面）

问题描述

使用 cn.csv 数据（中国 2577 个城市，包含经纬度、人口、行政级别等），从中选择行政级别为 admin 或 primary 的城市作为候选设施点，建立 p-Median 模型，选择 $p = 20\%$ 的城市作为设施点，最小化所有城市到最近设施点的总距离。结果用 folium 地图可视化。

```
import gurobipy as gp


# ===== 数据 =====
revenue = {
    1: 127, 2: 83, 3: 165, 4: 96, 5: 112,
    6: 88, 7: 135, 8: 141, 9: 117, 10: 94
}
```

```
circles = [
    [1, 2, 4],        # 圆圈1
    [3, 4, 5],        # 圆圈2
    [4, 5, 6, 7],     # 圆圈3
    [5, 6, 9],        # 圆圈4
    [6, 7, 8],        # 圆圈5
    [8, 9, 10],       # 圆圈6
    [5, 9]            # 圆圈7
]
I = list(range(1, 11))    # [
```

Unit 4：Facility Location

4.1：p-Median 问题

给定 n 个客户（集合 $\mathcal{I}$）和 m 个候选设施点（集合 $\mathcal{J}$），需要从中选出 p 个设施点开设，并将每个客户分配到一个已开设的设施点，使得总服务距离/成本最小。

参数： c_{ij}：客户 i 由设施 j 服务的距离/成本　　p：需开设的设施数量

目标函数： $\min \quad \sum_{i\in\mathcal{I}}\sum_{j\in\mathcal{J}} c_{ij}\,y_{ij}$

约束条件：

$$\sum_{j\in\mathcal{J}} y_{ij} = 1, \quad \forall i \in \mathcal{I} \qquad \text{（每个客户必须被服务）}$$

$$\sum_{j\in\mathcal{J}} x_j = p \qquad \text{（恰好开设 } p \text{ 个设施）}$$

$$y_{ij} \le x_j, \quad \forall i \in \mathcal{I}, j \in \mathcal{J} \qquad \text{（只能分配给已开设的设施）}$$

$$x_j \in \{0,1\}, \quad y_{ij} \in \{0,1\}$$

```
import numpy as np
import gurobipy as gp
# ===== 数据=====
n = 10   # 客户数
m = 8    # 候选设施数
p = 3    # 开设设施数

np.random.seed(42)
loc_cust = np.random.randint(0, 100, (n, 2))   # 客户坐标
loc_fac = np.random.randint(0, 100, (m, 2))    # 设施坐标

# 计算欧氏距离：直接由两个点I,j的坐标封装求距离
c = {(i, j): np.linalg.norm(loc_cust[i] - loc_fac[j])
     for i in range(n) for j in range(m)}

I = range(n)
J = range(m)

# ===== 模型建立 =====
model = gp.Model('pmedian')

x = model.addVars(J, vtype=gp.GRB.BINARY, name='x')
y = model.addVars(I, J, vtype=gp.GRB.BINARY, name='y')

model.setObjective(gp.quicksum(c[i, j] * y[i, j] for i in I for j
in J), gp.GRB.MINIMIZE)

model.addConstrs((gp.quicksum(y[i, j] for j in J) == 1 for i in I),
name='assign')
model.addConstr(gp.quicksum(x[j] for j in J) == p, name='p_facilit
ies')
model.addConstrs((y[i, j] <= x[j] for i in I for j in J), name='li
nk')

model.optimize()

# 输出结果
if model.status == gp.GRB.OPTIMAL:
    print(f"最优总距离 = {model.ObjVal:.2f}")
    opened = [j for j in J if x[j].X > 0.9]
    print(f"开设设施: {opened}")
    for i in I:
        assigned = [j for j in J if y[i, j].X > 0.9]
        print(f"  客户 {i} → 设施 {assigned[0]}, 距离={c[i, assigne
d[0]]:.2f}")
```

```
m = gp.Model('store_selection')

x = m.addVars(I, vtype=gp.GRB.BINARY, name='x')

m.setObjective(x.prod(revenue), gp.GRB.MAXIMIZE)
# x.prod(re)即sum（xi*re），"点积"
m.addConstrs(
    (gp.quicksum(x[i] for i in circles[k]) <= 1 fo
r k in range(7)),
    name='conflict'
) # 对于给出矩阵情况下的求解

# ===== 求解 =====
m.optimize()

print('=== 求解结果 ===')
keep = [i for i in I if x[i].X > 0.9]
close = [i for i in I if x[i].X < 0.1]
print(f'保留的门店：{keep}')
print(f'关闭的门店：{close}')
print(f'总收入：${m.objVal * 1000:,.0f}')
```

4.2：p-Center 问题（课件）

与 p-Median 类似，但目标改为最小化**最远的**客户到设施的距离，体现公平性。常用于消防站、急救中心等公共设施的选址。

数学建模

引入辅助变量 w 表示最大服务距离。

目标函数： $\min \quad w$

约束条件：

$$\sum_{j\in\mathcal{J}} y_{ij} = 1, \quad \forall i \in \mathcal{I}$$

$$\sum_{j\in\mathcal{J}} x_j = p$$

$$y_{ij} \le x_j, \quad \forall i, j$$

$$\sum_{j\in\mathcal{J}} c_{ij}\,y_{ij} \le w, \quad \forall i \in \mathcal{I} \qquad \text{（每个客户的服务距离不超过 } w\text{）}$$

$$x_j \in \{0,1\}, \quad y_{ij} \in \{0,1\}$$

```
import numpy as np
import gurobipy as gp

# ===== 数据=====
n = 10   # 客户数
m = 8    # 候选设施数
p = 3    # 开设设施数

np.random.seed(42)
loc_cust = np.random.randint(0, 100, (n, 2))
loc_fac = np.random.randint(0, 100, (m, 2))

c = {(i, j): np.linalg.norm(loc_cust[i] - loc_fac[j])
     for i in range(n) for j in range(m)}

I = range(n)
J = range(m)
# ===== 模型建立 =====
model = gp.Model('pcenter')

x = model.addVars(J, vtype=gp.GRB.BINARY, name='x')
y = model.addVars(I, J, vtype=gp.GRB.BINARY, name='y')
w = model.addVar(lb=0, name='w')   # 最大服务距离

model.setObjective(w, gp.GRB.MINIMIZE)

model.addConstrs((gp.quicksum(y[i, j] for j in J) == 1 for i in I),
name='assign')
model.addConstr(gp.quicksum(x[j] for j in J) == p, name='p_facilit
ies')
model.addConstrs((y[i, j] <= x[j] for i in I for j in J), name='li
nk')
model.addConstrs((gp.quicksum(c[i, j] * y[i, j] for j in J) <= w f
or i in I), name='max_dist')

model.optimize()
# 输出结果
if model.status == gp.GRB.OPTIMAL:
    print(f"最优最大距离 = {w.X:.2f}")
    opened = [j for j in J if x[j].X > 0.9]
    print(f"开设设施: {opened}")
    for i in I:
        assigned = [j for j in J if y[i, j].X > 0.9]
        dist = c[i, assigned[0]]
        print(f"  客户 {i} → 设施 {assigned[0]}, 距离={dist:.2f}")
```

4.4：作业 — 门店关闭选址（Grocery Store Chain）

某杂货连锁店在一个城市有 10 家门店，要求任意两家门店之间的距离不能小于 2 英里。下图将距离小于 2 英里的门店用圆圈标出。已知各门店月收益（单位：千美元），需要决定保留哪些门店，使得保留门店的总收益最大。

门店	1	2	3	4	5	6	7	8	9	10
收益	127	83	165	96	112	88	135	141	117	94

互斥圆圈（距离 < 2 英里）： 略

决策变量：

$x_i \in \{0,1\}$：门店 i 是否保留（1 = 保留）

目标函数： $\max \quad \sum_{i=1}^{10} r_i\,x_i$

其中 r_i 为门店 i 的收益。

约束条件：

$$x_1 + x_2 + x_4 \le 1 \qquad \text{(Circle 1)}$$

$$x_3 + x_4 + x_5 \le 1 \qquad \text{(Circle 2)}$$

$$x_4 + x_5 + x_6 + x_7 \le 1 \qquad \text{(Circle 3)}$$

$$x_5 + x_6 + x_9 \le 1 \qquad \text{(Circle 4)}$$

$$x_6 + x_7 + x_8 \le 1 \qquad \text{(Circle 5)}$$

$$x_8 + x_9 + x_{10} \le 1 \qquad \text{(Circle 6)}$$

$$x_5 + x_9 \le 1 \qquad \text{(Circle 7)}$$

$$x_i \in \{0,1\}, \quad i = 1,\ldots,10$$

```python
import numpy as np ### 4.3
import gurobipy as gp

# =====数据=====
n = 8   # 客户数
m = 5   # 候选设施数

np.random.seed(42)
d = np.random.randint(10, 50, n) # 客户需求
f = np.random.randint(100, 200, m) # 固定开设成本
v = np.random.randint(80, 150, m) # 设施容量
c = np.random.randint(1, 10, (n, m)) # 单位服务成本

I = range(n)
J = range(m)

# ===== 模型建立 =====
model = gp.Model('fixed_charge')

x = model.addVars(J, vtype=gp.GRB.BINARY, name='x')
y = model.addVars(I, J, lb=0, ub=1, name='y')

model.setObjective(
    gp.quicksum(f[j] * x[j] for j in J) +
    gp.quicksum(c[i, j] * d[i] * y[i, j] for i in I for j in J),
    gp.GRB.MINIMIZE
)

model.addConstrs((gp.quicksum(y[i, j] for j in J) == 1 for i in I), name='demand
')
model.addConstrs((gp.quicksum(d[i] * y[i, j] for i in I) <= v[j] * x[j] for j in
J), name='capacity')

model.optimize()

# 输出结果
if model.status == gp.GRB.OPTIMAL:
    print(f"最优总成本 = {model.ObjVal:.2f}")
    opened = [j for j in J if x[j].x > 0.9]
    print(f"开设设施: {opened}")
    for j in opened:
        served = [i for i in I if y[i, j].x > 0.01]
        print(f"  设施 {j} 服务客户: {served}")
```

```python
import numpy as np   ###4.5
import pandas as pd
import gurobipy as gp
import folium
# ===== 数据读取与预处理 =====
data = pd.read_csv('cn.csv') # 读取CSV文件，每行一个城市，含经纬度、行政级别等字段
data = data[(data['capital'] == 'admin') | (data['capital'] == 'primary')]
n = data.shape[0] # 获取筛选后的行数（城市数量）
cities = data['city'].values # 提取'city'列 → 城市名称数组
loc_x_vals = data['lat'].values # 纬度 # 提取'Lat'列 → 纬度数组（后续计算距离用）
loc_y_vals = data['lng'].values   # 经度 # 提取'Lng'列 → 经度数组（后续计算距离用）
loc_x = dict(zip(cities, loc_x_vals))
loc_y = dict(zip(cities, loc_y_vals))

I = cities   # 客户集合
J = cities   # 候选设施集合
p = round(n * 0.2)   # 开设设施数量 (20%)
# 计算欧氏距离（基于经纬度）
c = {(i, j): ((loc_x[i] - loc_x[j])**2 + (loc_y[i] - loc_y[j])**2)**0.5
     for i in I for j in J}
# ===== 模型建立 =====
m = gp.Model('pmedian')

x = m.addVars(I, vtype=gp.GRB.BINARY, name='x')       # 设施点是否开设
y = m.addVars(I, J, vtype=gp.GRB.BINARY, name='y')   # 客户分配给哪个设施

m.setObjective(y.prod(c), gp.GRB.MINIMIZE)

m.addConstrs((y.sum(i, '*') == 1 for i in I), name='cover')   # 每个客户被覆盖
m.addConstr(x.sum() == p, name='p_location') # 恰好开设 p 个
m.addConstrs((y[i, j] <= x[j] for i in I for j in J), name='capacity') # 只能去开设的设施
# ===== 求解 =====
m.params.TimeLimit = 30
m.optimize()

print('The solution time is', m.runtime)
print('The MIP gap is', m.mipgap)
# ===== 可视化 =====
if m.status != gp.GRB.INFEASIBLE:
    pmedian_map = folium.Map(location=[34.32, 106.55], zoom_start=4)

    cities_selected = [j for j in J if x[j].x > 0.9]
    for i in I:
        if i in cities_selected:
            folium.Marker(location=(loc_x[i], loc_y[i]),
                          icon=folium.Icon(color='red'), popup=i).add_to(pme
dian_map)
        else:
            folium.Marker(location=(loc_x[i], loc_y[i]), popup=i).add_to(pme
dian_map)
    # 绘制分配关系
    assignments = [(i, j) for i in I for j in J if y[i, j].x > 0.9]
    for (i, j) in assignments:
        folium.PolyLine(
            [(loc_x[i], loc_y[i]), (loc_x[j], loc_y[j])]
        ).add_to(pmedian_map)

    pmedian_map.save('pmedian_map.html')
```

Unit 5：Traveling Salesman Problem　5.1：TSP 基础模型

给定 n 个城市（节点集合 $\mathcal{N}=\{0,1,\dots,n-1\}$），以及每对城市之间的距离 c_{ij}，寻找一条最短的回路，使得每个城市恰好被访问一次并返回起点。

参数： c_{ij}：城市 i 到 j 的距离；$\mathcal{A}=\{(i,j)\mid i,j\in\mathcal{N}, i\neq j\}$：弧集合

决策变量：

$x_{ij}\in\{0,1\}$：是否从城市 i 直接走到城市 j

$u_i\in[1,n-1]$：城市 i 的访问顺序（MTZ辅助变量，$i=1,\dots,n-1$）

目标函数： $\min\ \sum_{(i,j)\in\mathcal{A}} c_{ij}\,x_{ij}$

$$\sum_{j:(j,i)\in\mathcal{A}} x_{ji}=1,\quad \forall i\in\mathcal{N}\qquad\text{（每个城市恰好进入一次）}$$

$$\sum_{j:(i,j)\in\mathcal{A}} x_{ij}=1,\quad \forall i\in\mathcal{N}\qquad\text{（每个城市恰好离开一次）}$$

约束条件：

$$u_j\geq u_i+1-n(1-x_{ij}),\quad \forall i,j\in\{1,\dots,n-1\}, i\neq j\qquad\text{（MTZ 消圈）}$$

$$x_{ij}\in\{0,1\},\quad 1\leq u_i\leq n-1$$

5.2：CVRP（带容量约束的车辆路径问题，课件）

问题描述

在 TSP 基础上，引入车辆容量限制。有 m 辆同质车辆，每辆容量为 Q。客户 i 的需求为 q_i。从仓库（节点 0）出发，需要规划车辆路径，在满足容量限制的前提下服务所有客户，总行驶成本最小。

参数：

q_i：客户 i 的需求；Q：车辆容量；m：可用车辆数；c_{ij}：弧 (i,j) 的行驶成本

决策变量：

$x_{ij}\in\{0,1\}$：弧 (i,j) 是否被某辆车使用

$u_i\in[q_i,Q]$：车辆到达客户 i 后的剩余装载量（用于消圈+容量）

目标函数： $\min\ \sum_{(i,j)\in\mathcal{A}} c_{ij}\,x_{ij}$

$$\sum_{j\in\mathcal{N}} x_{ij}=\sum_{j\in\mathcal{N}} x_{ji}=1,\quad \forall i\in\mathcal{N}_c\qquad\text{（每个客户恰好被访问一次）}$$

$$\sum_{j\in\mathcal{N}} x_{0j}\leq m\qquad\text{（从仓库出发的车辆数不超过}m\text{）}$$

约束条件：

$$u_j-u_i+Q(1-x_{ij})\geq q_j,\quad \forall i,j\in\mathcal{N}_c, i\neq j\qquad\text{（容量+消圈）}$$

$$q_i\leq u_i\leq Q,\quad \forall i\in\mathcal{N}_c$$

$$x_{ij}\in\{0,1\}$$

注：u_i 同时承担了 MTZ 消圈和车辆容量限制两个功能。

5.3：Orienteering Problem（定向问题，作业）

问题描述

TSP 的变种：：给定 n 个节点，每个节点有得分 s_i，从节点 0（仓库）出发并返回节点 0。每段弧 (i,j) 有旅行时间 c_{ij}，总旅行时间不能超过预算 T。不需要访问所有节点，目标是选择一部分节点访问并确定访问顺序，使得**收集的总得分最大**。

参数：

s_i：节点 i 的得分；c_{ij}：从节点 i 到 j 的旅行时间；T：时间预算；n：节点总数

决策变量：

$x_{ij}\in\{0,1\}$：是否从节点 i 直接走到节点 j

$y_i\in\{0,1\}$：是否访问节点 i

$u_i\in[0,n-1]$：节点 i 的访问顺序（MTZ辅助变量）

目标函数： $\max\ \sum_{i=0}^{n-1} s_i\,y_i$

约束条件：

$$y_0=1\qquad\text{（必须从节点 0 出发）}$$

$$\sum_j x_{ji}=\sum_j x_{ij}=y_i,\quad \forall i\in\mathcal{N}\qquad\text{（流守恒）}$$

$$\sum_{i,j} c_{ij}\,x_{ij}\leq T\qquad\text{（时间预算）}$$

$$(x_{ij}=1)\Rightarrow(u_j\geq u_i+1),\quad \forall i,j\in\{1,\dots,n-1\}, i\neq j\qquad\text{（MTZ 消圈）}$$

$$u_i\leq(n-1)y_i,\quad \forall i\in\{1,\dots,n-1\}\qquad\text{（}u\text{ 与 }y\text{ 关联，未访问节点 }u=0\text{）}$$

$$u_i\geq y_i,\quad \forall i\in\{1,\dots,n-1\}$$

$$x_{ij}\in\{0,1\},\quad y_i\in\{0,1\}$$

7.1：矩阵博弈与最优纯策略

问题描述

二人有限零和博弈：局中人甲和乙各有有限个策略，同一局势下两人损益之和为零。给定甲的赢得矩阵 $A=(a_{ij})_{m\times n}$，求双方最优策略。

前提假设： 双方都是理性的，都从各自最不利情形中选择最有利情形。

若满足：

$$\max_i\min_j a_{ij}=\min_j\max_i a_{ij}=V$$

则称博弈存在**鞍点（saddle point）**，对应的策略为**最优纯策略**，V 为博弈的值。

1、对矩阵 A 的每一行求最小值（甲的最不利情形）

2、在这些行最小值中取最大者：$\underline{V}=\max_i\min_j a_{ij}$（甲的max-min）

3、对矩阵 A 的每一列求最大值（乙的最不利情形）

4、在这些列最大值中取最小者：$\overline{V}=\min_j\max_i a_{ij}$（乙的min-max）

5、若 $\underline{V}=\overline{V}$，则存在纯策略解

7.2：矩阵博弈的混合策略（课件）

当纯策略解不存在（即 $\max_i\min_j a_{ij}\neq\min_j\max_i a_{ij}$）时，双方以一定概率随机选择策略，称为**混合策略**。求最优混合策略等价于求解一个线性规划问题。

数学建模（LP 方法）

设甲有 m 个策略，乙有 n 个策略。甲选择策略 i 的概率为 x_i，乙选择策略 j 的概率为 y_j。

甲的视角（最大化最不利情形下的期望收益）：

决策变量：

$x_i\geq 0$：甲选择策略 i 的概率

v：甲在最不利情形下的期望收益（博弈值）

标准的 LP 写法（无需 $v>0$ 假设）：

$$\max\ v\quad\text{s.t.}\quad \sum_{i=1}^m a_{ij}\,x_i-v\geq 0,\ \ j=1,\dots,n$$

$$\sum_{i=1}^m x_i=1$$

$$x_i\geq 0$$

乙的对偶问题（最小化最不利情形下的期望损失）

$$\min\ w\quad\text{s.t.}\quad \sum_{j=1}^n a_{ij}\,y_j-w\leq 0,\ \ i=1,\dots,m$$

$$\sum_{j=1}^n y_j=1$$

$$y_j\geq 0$$

对偶关系： 根据强对偶定理，$v^*=w^*$，即双方最优混合策略下的博弈值相同。

```python
import numpy as np ###5.1
import matplotlib.pyplot as plt
import gurobipy as gp
# ===== 数据=====
# 实际作业中应替换为题目提供的坐标或 CSV
n = 20
N = list(range(n))
A = [(i, j) for i in N for j in N if i != j]

np.random.seed(0)
loc_x = np.random.randint(0, 100, n)
loc_y = np.random.randint(0, 100, n)
# 计算欧氏距离
d = {(i, j): ((loc_x[i] - loc_x[j])**2 + (loc_y[i] - loc_y[j])**2)**0.5
     for (i, j) in A}
# ===== 模型建立 =====
m = gp.Model('tsp')

x = m.addVars(A, vtype=gp.GRB.BINARY, name='x')

m.setObjective(x.prod(d), gp.GRB.MINIMIZE)
# 流守恒约束
m.addConstrs((x.sum('*', i) == 1 for i in range(n)), name='inflow')
m.addConstrs((x.sum(i, '*') == 1 for i in range(n)), name='outflow')
# MTZ 消圈约束
u = m.addVars(N, lb=1, ub=n-1, name='u')
m.addConstrs(
    ((x[i, j] == 1) >> (u[j] >= u[i] + 1)
     for i in range(1, n) for j in range(1, n) if i != j),
    name='mtz'
)
# ===== 求解 =====
m.params.timelimit = 30
m.optimize()
# ===== 可视化 =====
if m.status != gp.GRB.INFEASIBLE:
    selected = [(i, j) for (i, j) in A if x[i, j].x > 0.9]
    plt.scatter(loc_x, loc_y, c='r', marker='o') #用红色圆点标出每个城市的位置
    for (i, j) in selected:
        plt.plot((loc_x[i], loc_x[j]), (loc_y[i], loc_y[j]), c='b') #蓝色连线
    plt.savefig('tour.png', dpi=300)
    plt.show()
```

```python
import numpy as np ###5.2
import gurobipy as gp
# =====数据=====
n = 15          # 客户数
Q = 50          # 车辆容量
m_vehicles = 3  # 可用车辆数

np.random.seed(42)
loc = np.random.randint(0, 100, (n+1, 2))   # 节点 0 是仓库
q = np.random.randint(5, 20, n)             # 客户需求

c = {(i, j): np.linalg.norm(loc[i] - loc[j]) for i in range(n+1) for j in ra
nge(n+1) if i != j}

N = list(range(n+1))
A = [(i, j) for i in N for j in N if i != j]
Nc = list(range(1, n+1))   # 客户节点
# ===== 模型建立 =====
model = gp.Model('cvrp')

x = model.addVars(A, vtype=gp.GRB.BINARY, name='x')
u = model.addVars(Nc, lb=0, ub=Q, name='u')

model.setObjective(gp.quicksum(c[i, j] * x[i, j] for i, j in A), gp.GRB.MINI
MIZE)
# 流守恒
model.addConstrs((gp.quicksum(x[i, j] for j in N if j != i) == 1 for i in N
c), name='outflow')
model.addConstrs((gp.quicksum(x[j, i] for j in N if j != i) == 1 for i in N
c), name='inflow')
# 从仓库出发的车辆数
model.addConstr(gp.quicksum(x[0, j] for j in Nc) <= m_vehicles, name='vehicl
es')
# 容量 + MTZ 消圈
model.addConstrs(
    (u[j] - u[i] + Q * (1 - x[i, j]) >= q[j-1]
     for i in Nc for j in Nc if i != j),
    name='capacity_mtz'
)
# 初始装载量
model.addConstrs((u[i] >= q[i-1] for i in Nc), name='u_lb')

model.params.TimeLimit = 30
model.optimize()
# 输出结果
if model.status != gp.GRB.INFEASIBLE:
    print(f"最优总距离 = {model.ObjVal:.2f}")
    routes = [(i, j) for i, j in A if x[i, j].X > 0.9]
    print(f"使用弧数: {len(routes)}")
    # 按路径分组输出
    from collections import defaultdict
    out_edges = defaultdict(list)
    for i, j in routes:
        out_edges[i].append(j)
    print("
各路径: ")
    for start in out_edges[0]:
        route = [0, start]
        cur = start
        while cur != 0:
            cur = out_edges[cur][0]
            route.append(cur)
        print(f"  {' → '.join(map(str, route))}")
```

```python
import numpy as np  ###5.3
import matplotlib.pyplot as plt
import gurobipy as gp
# ===== 数据 =====
# 方式1：直接导入（若在同一目录）
exec(open('op_random_instance.py').read())  # 执行该文件，把n/loc_x/loc_y/s/c/T等变量读入当前环境
N = list(range(n))
A = [(i, j) for i in N for j in N if i != j]
# ===== 模型建立 =====
m = gp.Model('op')

x = m.addVars(A, vtype=gp.GRB.BINARY, name='x')   # 弧是否被使用
y = m.addVars(N, vtype=gp.GRB.BINARY, name='y')    # 节点是否被访问
u = m.addVars(N, lb=0, ub=n-1, name='u')           # MTZ 顺序变量
# 目标：最大化总得分
m.setObjective(gp.quicksum(s[i] * y[i] for i in N), gp.GRB.MAXIMIZE)
# 约束1：必须经过起点（节点 0）
m.addConstr(y[0] == 1, name='visit_depot')
# 约束2：流守恒（入=出=y_i）
m.addConstrs((x.sum('*', i) == y[i] for i in N), name='inflow')
m.addConstrs((x.sum(i, '*') == y[i] for i in N), name='outflow')
# 约束3：时间预算
m.addConstr(gp.quicksum(c[i, j] * x[i, j] for i, j in A) <= T, name='time_budget
')
# 约束4：MTZ 消圈（仅非仓库节点）
m.addConstrs(
    ((x[i, j] == 1) >> (u[j] >= u[i] + 1)
     for i in range(1, n) for j in range(1, n) if i != j),
    name='mtz'
)
# 约束5：u 与 y 挂钩（未访问节点 u=0，访问节点 u>=1）
m.addConstrs((u[i] <= (n - 1) * y[i] for i in range(1, n)), name='u_link_y')
m.addConstrs((u[i] >= y[i] for i in range(1, n)), name='u_lb')
# ===== 求解 =====
m.params.timelimit = 30
m.optimize()
# ===== 结果输出与可视化 =====
if m.status == gp.GRB.OPTIMAL or m.status == gp.GRB.TIME_LIMIT:
    visited = [i for i in N if y[i].x > 0.9]
    selected = [(i, j) for i, j in A if x[i, j].x > 0.9]
    total_score = sum(s[i] for i in visited)
    total_time = sum(c[i, j] for i, j in selected)

    print(f"Visited nodes: {visited}")
    print(f"Total collected score: {total_score}")
    print(f"Total travel time: {total_time:.2f} / {T}")
    # 绘图
    plt.figure(figsize=(8, 8))
    plt.scatter(loc_x, loc_y, c='lightgray', s=100, zorder=1, label='Unvisited')
    plt.scatter(loc_x[visited], loc_y[visited], c='red', s=200, zorder=3, label='
Visited')
    for i in N:
        color = 'red' if i in visited else 'gray'
        plt.annotate(f'{i}\n(s={s[i]})', (loc_x[i], loc_y[i]),
                     textcoords="offset points", xytext=(0, 10),
                     ha='center', fontsize=8, color=color)
    for (i, j) in selected:
        plt.plot((loc_x[i], loc_x[j]), (loc_y[i], loc_y[j]),
                 c='blue', linewidth=2, zorder=2)
    plt.title(f'Orienteering Problem\nScore: {total_score}, Time: {total_time:.2
f}/{T}')
    plt.legend()
    plt.savefig('op_tour.png', dpi=300)
```

```python
import numpy as np  ###7.2
import gurobipy as gp

# ===== 赢得矩阵（甲的视角）=====
A = np.array([
    [5, 9],
    [8, 6]
])

m, n = A.shape
# ===== 甲的 LP：max v =====
model_a = gp.Model('game_a')

x = model_a.addVars(m, lb=0, name='x')
v = model_a.addVar(lb=-gp.GRB.INFINITY, name='v')

model_a.setObjective(v, gp.GRB.MAXIMIZE)
# 无论乙选什么策略，甲至少得 v
for j in range(n):
    model_a.addConstr(gp.quicksum(A[i, j] * x[i] for i in range
(m)) >= v, name=f'strategy_{j}')

model_a.addConstr(gp.quicksum(x[i] for i in range(m)) == 1, name='
prob')

model_a.optimize()

print("=== 甲（行玩家）最优混合策略 ===")
if model_a.status == gp.GRB.OPTIMAL:
    print(f"博弈值 v = {v.X:.4f}")
    for i in range(m):
        print(f"  x_{i+1} = {x[i].X:.4f}")
```

表格	
甲 (Primal)	乙 (Dual)
$\max v$	$\min w$
变量：m 个 x_i+1 个 v	变量：n 个 y_j+1 个 w
约束：n 个 $\geq+1$ 个 $=$	约束：m 个 $\leq+1$ 个 $=$
右端项：v（目标系数）	右端项：w（目标系数）